Computer Network Lab Answers

15 practical programs with simple Java code and sample outputs

For programs that use two endpoints, start the server first and then run the client in another terminal. Network command results and DNS addresses can vary by system.

1. Network Diagnosis Using Commands

Aim

Check connectivity, active connections, DNS resolution, and packet routes.

Algorithm

1. Run ping to test connectivity.
2. List active connections.
3. Look up a domain name.
4. Trace the route to the host.

Code

```
import java.io.*;

public class NetworkDiagnosis {
    static void run(String... cmd) throws Exception {
        Process p = new ProcessBuilder(cmd).redirectErrorStream(true).start();
        p.getInputStream().transferTo(System.out);
        p.waitFor();
    }

    public static void main(String[] args) throws Exception {
        boolean win = System.getProperty("os.name").toLowerCase().contains("win");
        run("ping", win ? "-n" : "-c", "2", "google.com");
        run("nslookup", "google.com");
        run(win ? "tracert" : "traceroute", "google.com");
        run(win ? "netstat" : "netstat", "-an");
    }
}
```

Output

Ping replies, DNS address, route hops, and active connections are displayed.

Result

Connectivity, DNS, routes, and active connections were checked.

2. Download a Web Page Using TCP

Aim

Connect to a web server over TCP, send an HTTP request, and display the response.

Algorithm

1. Create a TCP connection to the web server.
2. Send an HTTP GET request.
3. Read and display the response.
4. Close the connection.

Code

```
import java.io.*;
import java.net.*;

public class WebClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("example.com", 80);
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
        out.print("GET / HTTP/1.0\r\nHost: example.com\r\n\r\n");
        out.flush();
        String line;
        while ((line = in.readLine()) != null) System.out.println(line);
        s.close();
    }
}
```

Output

```
HTTP/1.0 200 OK
Content-Type: text/html
<!doctype html>...
```

Result

The client connected to the web server and received its page.

3. TCP Message Echo

Aim

Send a message from a client to a server and return the same message.

Algorithm

1. Start the server on a TCP port.
2. Connect the client to the server.
3. Send a message and read it at the server.
4. Return the message to the client.

Code

```
import java.io.*;
import java.net.*;

public class TcpEcho {
    public static void main(String[] args) throws Exception {
        if (args[0].equals("server")) {
            ServerSocket ss = new ServerSocket(5000);
            Socket s = ss.accept();
            BufferedReader in = new BufferedReader(
                    new InputStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(s.getOutputStream(), true);
            out.println(in.readLine());
            s.close(); ss.close();
        } else {
            Socket s = new Socket("localhost", 5000);
            PrintWriter out = new PrintWriter(s.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                    new InputStreamReader(s.getInputStream()));
            out.println("Hello Server");
            System.out.println(in.readLine());
            s.close();
        }
    }
}
```

Output

Hello Server

Result

The server received and returned the client's message.

4. TCP Chat Application

Aim

Exchange messages between a client and server using TCP.

Algorithm

1. Start the server and wait for a client.
2. Connect the client to the server.
3. Send and receive messages in turns.
4. End the chat when either side sends bye.

Code

```
import java.io.*;
import java.net.*;

public class TcpChat {
    public static void main(String[] args) throws Exception {
        boolean server = args[0].equals("server");
        Socket s;
        if (server) s = new ServerSocket(5001).accept();
        else s = new Socket("localhost", 5001);
        BufferedReader net = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
        BufferedReader key = new BufferedReader(new InputStreamReader(System.in));
        while (true) {
            if (!server) {
                System.out.print("You: ");
                String m = key.readLine(); out.println(m);
                if (m.equals("bye")) break;
            }
            String m = net.readLine();
            System.out.println((server ? "Client: " : "Server: ") + m);
            if (m == null || m.equals("bye")) break;
            if (server) {
                System.out.print("You: ");
                m = key.readLine(); out.println(m);
                if (m.equals("bye")) break;
            }
        }
        s.close();
    }
}
```

Output

```
You: Hello
Server: Hi
```

Result

The client and server exchanged messages over TCP.

5. Simplified DNS Service Using UDP

Aim

Resolve a domain name to an IP address using a UDP client-server program.

Algorithm

1. Start a UDP server on a port.
2. Receive a domain name from the client.
3. Resolve the domain to an IP address.
4. Send the address back to the client.

Code

```
import java.net.*;

public class UdpDns {
    public static void main(String[] args) throws Exception {
        if (args[0].equals("server")) {
            DatagramSocket s = new DatagramSocket(5002);
            byte[] b = new byte[100];
            DatagramPacket p = new DatagramPacket(b, b.length);
            s.receive(p);
            String domain = new String(p.getData(), 0, p.getLength());
            String ip = InetAddress.getByName(domain).getHostAddress();
            byte[] reply = ip.getBytes();
            s.send(new DatagramPacket(reply, reply.length,
                    p.getAddress(), p.getPort()));
            s.close();
        } else {
            DatagramSocket s = new DatagramSocket();
            byte[] b = "example.com".getBytes();
            s.send(new DatagramPacket(b, b.length,
                    InetAddress.getByName("localhost"), 5002));
            byte[] r = new byte[100];
            DatagramPacket p = new DatagramPacket(r, r.length);
            s.receive(p);
            System.out.println(new String(p.getData(), 0, p.getLength()));
            s.close();
        }
    }
}
```

Output

93.184.216.34 (example; actual address may differ)

Result

The UDP server resolved the domain and returned its IP address.

6. Capture and Analyze Network Traffic

Aim

Capture packets and inspect their addresses, ports, protocols, and headers using Wireshark.

Algorithm

1. Open Wireshark and select the active interface.
2. Start capturing packets.
3. Run the Java program to generate network traffic.
4. Stop the capture and inspect packet headers.

Code

```
import java.io.*;

public class GenerateTraffic {
    public static void main(String[] args) throws Exception {
        boolean win = System.getProperty("os.name").toLowerCase().contains("win");
        Process p = new ProcessBuilder("ping", win ? "-n" : "-c", "2",
                "example.com").redirectErrorStream(true).start();
        p.getInputStream().transferTo(System.out);
        p.waitFor();
    }
}
```

Output

Ping replies appear in the terminal. Wireshark shows source/destination IPs and ICMP packets.

Result

Traffic was generated and its packet fields can be inspected in Wireshark.

7. Capture Ping and Traceroute Traffic

Aim

Generate ping and traceroute packets and examine them using a protocol analyzer.

Algorithm

1. Start a Wireshark capture.
2. Run ping and traceroute to a host.
3. Stop the capture after the commands finish.
4. Filter for ICMP and inspect packet details.

Code

```
import java.io.*;

public class TraceTraffic {
    static void run(String... cmd) throws Exception {
        Process p = new ProcessBuilder(cmd).redirectErrorStream(true).start();
        p.getInputStream().transferTo(System.out);
        p.waitFor();
    }
    public static void main(String[] args) throws Exception {
        boolean win = System.getProperty("os.name").toLowerCase().contains("win");
        run("ping", win ? "-n" : "-c", "2", "example.com");
        run(win ? "tracert" : "traceroute", "example.com");
    }
}
```

Output

Ping replies and traceroute hops are printed; Wireshark captures ICMP traffic.

Result

Ping and route packets were generated for analysis.

8. IP to MAC Address Mapping Using ARP

Aim

Display the local ARP table to observe IP-to-MAC address mappings.

Algorithm

1. Generate network traffic to a local host or gateway.
2. Run the operating system ARP command.
3. Find the IP address and its corresponding MAC address.
4. Optionally inspect ARP request/reply packets in Wireshark.

Code

```
import java.io.*;

public class ArpTable {
    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("arp", "-a")
                .redirectErrorStream(true).start();
        p.getInputStream().transferTo(System.out);
        p.waitFor();
    }
}
```

Output

```
Interface: 192.168.1.5
Internet Address      Physical Address
192.168.1.1           aa-bb-cc-dd-ee-ff
```

Result

The ARP table displayed IP addresses and their associated MAC addresses.

9. Congestion Control Demonstration

Aim

Model a limited-capacity router queue and observe packet drops when it is full.

Algorithm

1. Set a maximum queue capacity.
2. Generate packets faster than the queue can hold.
3. Count packets accepted and dropped.
4. Display the results.

Code

```
import java.util.*;

public class CongestionDemo {
    public static void main(String[] args) {
        Queue<Integer> q = new ArrayDeque<>();
        int capacity = 5, dropped = 0;
        for (int i = 1; i <= 10; i++) {
            if (q.size() == capacity) dropped++;
            else q.add(i);
        }
        System.out.println("Packets queued: " + q.size());
        System.out.println("Packets dropped: " + dropped);
    }
}
```

Output

```
Packets queued: 5
Packets dropped: 5
```

Result

The queue model demonstrated packet loss when the buffer was full.

10. Compare TCP and UDP Transmission

Aim

Compare delivery, delay, throughput, and packet loss in a simple model under identical conditions.

Algorithm

1. Set a common number of packets and link conditions.
2. Model TCP retransmission of lost packets.
3. Model UDP delivery without retransmission.
4. Display delivery, delay, throughput, and loss.

Code

```
public class TcpUdpCompare {
    public static void main(String[] args) {
        int sent = 100, lost = 10, packetBits = 8000;
        double seconds = 2.0;
        int tcpReceived = sent;
        int udpReceived = sent - lost;
        System.out.println("TCP: received=" + tcpReceived
                + ", loss=0, throughput=" + (tcpReceived * packetBits / seconds) + " bps");
        System.out.println("UDP: received=" + udpReceived
                + ", loss=" + lost + ", throughput=" + (udpReceived * packetBits / seconds) + " bps");
        System.out.println("Modeled delay: TCP=120 ms, UDP=80 ms");
    }
}
```

Output

```
TCP: received=100, loss=0, throughput=400000 bps
UDP: received=90, loss=10, throughput=360000 bps
Modeled delay: TCP=120 ms, UDP=80 ms
```

Result

The simplified model compared TCP and UDP using the same packet and time assumptions.

11. Distance Vector Routing

Aim

Find the least-cost routes between nodes using repeated distance-vector updates.

Algorithm

1. Initialize direct link costs and set other routes to infinity.
2. Exchange route costs between neighboring nodes.
3. Update a route when a lower cost is found.
4. Repeat until no route changes.

Code

```
import java.util.Arrays;


public class DistanceVector {
    public static void main(String[] args) {
        int inf = 999;
        int[][] d = {{0,2,7,inf},{2,0,1,5},{7,1,0,2},{inf,5,2,0}};
        for (int k = 0; k < 4; k++)
            for (int i = 0; i < 4; i++)
                for (int j = 0; j < 4; j++)
                    d[i][j] = Math.min(d[i][j], d[i][k] + d[k][j]);
        for (int[] row : d) System.out.println(Arrays.toString(row));
    }
}
```

Output

```
[0, 2, 3, 5]
[2, 0, 1, 3]
[3, 1, 0, 2]
[5, 3, 2, 0]
```

Result

The shortest route costs between all node pairs were calculated.

12. Link State Routing

Aim

Find optimal routes from a source node using link costs and Dijkstra's algorithm.

Algorithm

1. Represent the network as a weighted graph.
2. Set source distance to zero and other distances to infinity.
3. Visit the unvisited node with the lowest known cost.
4. Update neighbor costs and repeat.

Code

```
import java.util.*;

public class LinkState {
    public static void main(String[] args) {
        int inf = 999;
        int[][] g = {{0,2,7,inf},{2,0,1,5},{7,1,0,2},{inf,5,2,0}};
        int[] d = {0,inf,inf,inf};
        boolean[] used = new boolean[4];
        for (int c = 0; c < 4; c++) {
            int u = -1;
            for (int i = 0; i < 4; i++)
                if (!used[i] && (u == -1 || d[i] < d[u])) u = i;
            used[u] = true;
            for (int v = 0; v < 4; v++)
                d[v] = Math.min(d[v], d[u] + g[u][v]);
        }
        System.out.println(Arrays.toString(d));
    }
}
```

Output

[0, 2, 3, 5] (distances from node 0)

Result

The shortest paths from the source node were found using link costs.

13. Compare Distance Vector and Link State

Aim

Compare route costs found by Distance Vector and Link State algorithms on the same topology.

Algorithm

1. Use the same weighted network for both algorithms.
2. Calculate routes using repeated distance updates.
3. Calculate routes using Dijkstra's algorithm.
4. Compare route costs and update methods.

Code

```
import java.util.Arrays;

public class RoutingCompare {
    public static void main(String[] args) {
        int inf = 999;
        int[][] cost = {{0,2,7,inf},{2,0,1,5},{7,1,0,2},{inf,5,2,0}};
        int[][] dv = new int[4][4];
        for (int i = 0; i < 4; i++) dv[i] = cost[i].clone();
        for (int round = 0; round < 4; round++) {
            int[][] next = new int[4][4];
            for (int i = 0; i < 4; i++) next[i] = dv[i].clone();
            for (int i = 0; i < 4; i++)
                for (int j = 0; j < 4; j++)
                    for (int k = 0; k < 4; k++)
                        next[i][j] = Math.min(next[i][j], cost[i][k] + dv[k][j]);
            dv = next;
        }
        int[] ls = {0, inf, inf, inf};
        boolean[] used = new boolean[4];
        for (int c = 0; c < 4; c++) {
            int u = -1;
            for (int i = 0; i < 4; i++)
                if (!used[i] && (u == -1 || ls[i] < ls[u])) u = i;
            used[u] = true;
            for (int v = 0; v < 4; v++)
                ls[v] = Math.min(ls[v], ls[u] + cost[u][v]);
        }
        System.out.println("Distance Vector from A: " + Arrays.toString(dv[0]));
        System.out.println("Link State from A:      " + Arrays.toString(ls));
    }
}
```

Output

```
Distance Vector from A: [0, 2, 3, 5]
Link State from A:      [0, 2, 3, 5]
```

Result

Both algorithms found the same shortest route costs; their route update methods differ.

14. Cyclic Redundancy Check

Aim

Generate a CRC code for data and verify whether the received codeword has an error.

Algorithm

1. Append zeros to the data based on the generator length.
2. Divide using XOR operations to find the CRC remainder.
3. Append the remainder to the original data.
4. Divide the codeword; a zero remainder indicates no detected error.

Code

```
public class CrcDemo {
    static String remainder(String bits, String gen) {
        char[] a = bits.toCharArray();
        for (int i = 0; i <= a.length - gen.length(); i++)
            if (a[i] == '1')
                for (int j = 0; j < gen.length(); j++)
                    a[i+j] = a[i+j] == gen.charAt(j) ? '0' : '1';
        return new String(a, a.length - gen.length() + 1, gen.length() - 1);
    }
    public static void main(String[] args) {
        String data = "1101", gen = "1011";
        String crc = remainder(data + "000", gen);
        String code = data + crc;
        System.out.println("CRC: " + crc);
        System.out.println("Codeword: " + code);
        System.out.println("Check remainder: " + remainder(code, gen));
    }
}
```

Output

```
CRC: 001
Codeword: 1101001
Check remainder: 000
```

Result

The CRC was generated and the codeword passed the error check.

15. Detect Transmission Error Using CRC

Aim

Introduce an error into transmitted data and detect it using CRC at the receiver.

Algorithm

1. Generate a CRC and append it to the data.
2. Change one bit to simulate a transmission error.
3. Divide the received bits by the generator using XOR.
4. Report an error when the remainder is nonzero.

Code

```
public class CrcErrorDemo {
    static String remainder(String bits, String gen) {
        char[] a = bits.toCharArray();
        for (int i = 0; i <= a.length - gen.length(); i++)
            if (a[i] == '1')
                for (int j = 0; j < gen.length(); j++)
                    a[i+j] = a[i+j] == gen.charAt(j) ? '0' : '1';
        return new String(a, a.length - gen.length() + 1, gen.length() - 1);
    }
    public static void main(String[] args) {
        String data = "1101", gen = "1011";
        String sent = data + remainder(data + "000", gen);
        String received = (sent.charAt(0) == '0' ? "1" : "0") + sent.substring(1);
        System.out.println("Sent: " + sent);
        System.out.println("Received: " + received);
        System.out.println("Error detected: " + !remainder(received, gen).equals("000"));
    }
}
```

Output

```
Sent: 1101001
Received: 0101001
Error detected: true
```

Result

The receiver detected the simulated transmission error using CRC.